

Sun'iy intellekt agentlarini yaratish

Tabiiy tilni qayta ishlash — 15-ma'ruza

A.A. Abdulali

11:30–12:50 · mavzu №15

9-iyul 2026

Prezentatsiya rejasi

- 1 Kirish: LLM dan agentga
- 2 Agent va ReAct sikli
- 3 Vositalar va funksiya chaqirish
- 4 NLP modelini API qilish
- 5 Xavfsizlik va LangChain
- 6 Xulosa va keyingi ma'ruzaga ko'prik

Savol

L14 da sodda RAG pipeline ni ko'rdik:

qidirish → kontekstni promptga qo'shish → javob yaratish.

Shu pipeline qaysi vositani **qachon** ishlatishni va keyingi qadamni **o'zi** hal qiladimi?

O'ylab ko'ring: RAG faqat qidiruv pipeline mi yoki boshqaruvchi agentmi?

Takrorlash: L14 — RAGdan agentga ko'prik

Savol

RAG: **qidirish** → kontekstni promptga **qo'shish** → javob **yaratish**.

Shu pipeline qaysi vositani **qachon** ishlatishni va keyingi qadamni **o'zi** hal qiladimi?

Javob

Yo'q. L14 dagi sodda RAG — oldindan belgilangan **qidiruv** + **javob** pipeline. U mustaqil **reja tuzmaydi**, **tool tanlamaydi** va ko'p qadamli vazifani boshqarmaydi.

L14: RAG tashqi bilimdan kontekst topib, javobni asoslashga yordam beradi. **L15:** agent maqsadga qarab **rejalashtiradi**, kerakli **vositani (tool)** tanlaydi, natijani kuzatadi va keyingi qadamni hal qiladi. RAG esa agent chaqirishi mumkin bo'lgan vositalardan biriga aylanadi.

Bugungi savol: ko'p bosqichli vazifani kim boshqaradi?

Muammo

Vazifa bitta javob bilan tugamaydi: *“hujjatni topish, qisqacha mazmunini chiqarish, so'ng sentimentini baholash.”* Bunday so'rov **bir nechta ketma-ket amal** talab qiladi.

Vazifa ko'p amal talab qilsa:

- qaysi vosita avval chaqiriladi?
- qaysi natijadan keyin keyingi qadam tanlanadi?
- kim “qidir → qisqartir → bahola” tartibini boshqaradi?

Yechim

LLMga **agent roli** beriladi: u vazifani rejalashtiradi, mos **vosita (tool)**ni tanlaydi, natijani kuzatadi va keyingi qadamni hal qiladi.

Asosiy g'oya: agent faqat javob yozmaydi; u harakatlar ketma-ketligini boshqaradi.

Oddiy LLM va agent: asosiy farq

Toolsiz oddiy LLM

Foydalanuvchi matn kiritadi → model **matnli javob** yaratadi. Tashqi vosita/API chaqirmaydi va natijaga qarab yangi qadam tanlamaydi. Odatda **bir o'tishda** javob beradi.

Agent

Maqsadni tahlil qiladi → mos **vosita (tool)**ni tanlaydi → chaqiradi → natijani **kuzatuv** sifatida oladi → keyingi qadamni hal qiladi.

Oddiy LLM asosan javob matnini generatsiya qiladi. **Agent** esa javob berishdan oldin yoki javob jarayonida **rejalashtirish + tool chaqirish + kuzatuv + davom ettirish** siklidan foydalanadi.

Keyingi slaydda agentning tarkibiy qismlarini aniq ajratamiz.

Agent ta'rifi:

Agent $\approx$ LLM + vositalar (tools) + qadamlar tarixi (history) + boshqaruv sikli (control loop).

1. LLM — qaror yadrosi

Vazifani talqin qiladi, keyingi qadamni rejalashtiradi va qaysi amal bajarilishini tanlaydi.

2. Vositalar (tools) — bajaruvchi modul

Qidiruv, kalkulyator, tashqi API yoki NLP modelini chaqirib, real natija qaytaradi.

3. Qadamlar tarixi — kontekst

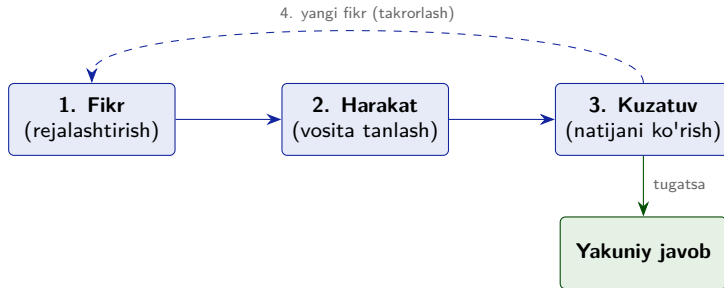
Oldingi harakatlar va kuzatuvlar saqlanadi: agent keyingi qarorni shu tarixga qarab chiqaradi.

4. Boshqaruv sikli — tartib

Fikr $\rightarrow$ harakat $\rightarrow$ kuzatuv jarayonini takrorlaydi va qachon to'xtashni nazorat qiladi.

Eslatma: bu yerda **xotira** deganda asosan joriy vazifadagi qadamlar tarixi nazarda tutiladi; uzoq muddatli memory alohida komponent bo'lishi mumkin.

Agent asosiy sikli: fikr → harakat → kuzatuv



- **Fikr:** LLM hozirgi holatga qarab keyingi qadamni o'ylaydi.
- **Harakat:** mos vositani tanlab chaqiradi.
- **Kuzatuv:** vosita qaytargan natijani oladi.
- **Yangi fikr:** natijaga qarab davom etadi yoki tugatadi.

Bu sikl agent yuragi. Keyingi bo'limda uni **ReAct** sifatida rasmiylashtiramiz.

Dars yakunida nimalarni bajara olasiz?

- ➊ Agent nima ekanini va u toolsiz oddiy LLMdan nimasi bilan farq qilishini **tushuntira olasiz**.
- ➋ ReAct(Reasoning and Acting) siklini a_t , o_t , h_t belgilari orqali **rasmiy ifodalay olasiz**.
- ➌ Vosita tanlashni va funksiya chaqirishni (*function-calling*) JSON orqali **tushuntira olasiz**.
- ➍ NLP modelini API sifatida ochib, uni agentga **vosita (tool)** sifatida ulash g'oyasini **loyihalashtira olasiz**.
- ➎ Agent xavfsizligi chegaralari (*guardrails*) va LangChainning agent arxitekturasidagi rolini **sanab bera olasiz**.

L14 dan: RAG, ya'ni qidiruv va javob yaratish pipeline. **L11–L12 dan:** LLM, attention va Transformer asoslari. **Dasturlashdan:** funksiya, JSON va HTTP so'rovi tushunchalari.

Bugungi dars rejasi va kutilgan natija

Bo'lim	Mavzu	Asosiy natija
1	Agent g'oyasi va asosiy sikl	Oddiy LLM va agent farqi
2	ReAct: fikrlash + harakat	a_t , o_t , h_t orqali sikl
3	Vositalar va funksiya chaqirish	Strukturali JSON chaqiruv
4	NLP modelini API orqali xizmatga chiqarish	POST /predict protokoli
5	Xavfsizlik va LangChain	Guardrails va agent framework
	Xulosa va 16-ma'ruzaga ko'prik	MLOps: deploy va monitoring

Bugungi darsning oqimi

Avval agent nima uchun kerakligini ko'ramiz; keyin ReAct siklini rasmiylashtiramiz; so'ng tool, API va LangChain orqali agent arxitekturasini to'liq zanjirga yig'amiz.

Prezentatsiya rejasi

- 1 Kirish: LLM dan agentga
- 2 Agent va ReAct sikli
- 3 Vositalar va funksiya chaqirish
- 4 NLP modelini API qilish
- 5 Xavfsizlik va LangChain
- 6 Xulosa va keyingi ma'ruzaga ko'prik

Intuitsiya: ReAct = mulohaza + harakat

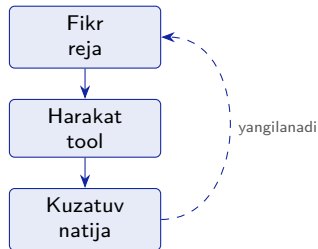
ReAct (*Reasoning + Acting*) — model keyingi qadamni **rejalashtiradi**, kerak bo'lsa **vosita (tool)** chaqiradi, natijani **kuzatadi** va shu natijaga qarab davom etadi. Oddiy intuitsiya:

“Avval nima qilish kerakligini aniqlayman, keyin bajaraman, natijaga qarab keyingi qadamni tanlayman.”

Eslatma: bu yerda “fikrlash” deganda foydalanuvchiga ko'rsatiladigan uzun ichki mulohaza emas, agentning keyingi qadamni tanlash jarayoni nazarda tutiladi.

Nega ReAct kerak?

Faqat reja — natijasiz. Faqat harakat — ko'r-ko'rona. ReAct ikkalasini navbatlashtiradi: har bir tool natijasi keyingi qarorni yangilaydi.

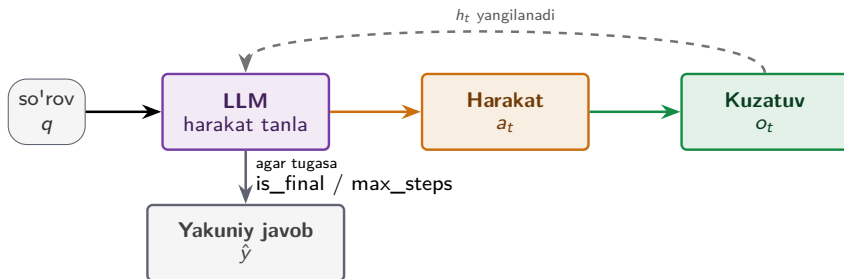


Keyingi slaydda shu intuitiv siklni a_t , o_t , h_t belgilarida rasmiylashtiramiz.

ReAct ta'rifi va ish jarayoni

ReAct — agentning har bir qadamda **harakat** tanlashi, vosita natijasini **kuzatishi**, shu natijani **tarixga qo'shib**, keyingi qarorni yangilab borish siklidir.

$$a_t = \text{LLM}(q, h_{t-1}), \quad o_t = \text{tool}(a_t), \quad h_t = h_{t-1} \parallel [(a_t, o_t)]$$



Tarix: $h_0 = [] \rightarrow h_1 = [(a_1, o_1)] \rightarrow h_2 = [(a_1, o_1), (a_2, o_2)]$

ReAct formulasi

$$a_t = \text{LLM}(q, h_{t-1}), \quad o_t = \text{tool}(a_t), \quad h_t = h_{t-1} \parallel [(a_t, o_t)].$$

q	foydalanuvchi so'rovi yoki vazifa;
h_{t-1}	$t - 1$ qadamgacha yig'ilgan tarix: oldingi harakatlar va kuzatuvlar;
a_t	t -qadamdagi harakat: LLM tanlagan tool nomi + argumentlar ;
$\text{tool}(a_t)$	tanlangan vositani bajarish jarayoni;
o_t	vosita qaytargan kuzatuv/natija;
h_t	yangilangan tarix; $\parallel$ belgisi ulash, ya'ni tarixga yangi (a_t, o_t) juftligini qo'shishni bildiradi.

To'xtash sharti

Agar LLM yakuniy javob yetarli deb topsa, sikl `is_final` bilan to'xtaydi va $\hat{y}$ qaytariladi. Aks holda keyingi qadamga o'tiladi; `max_steps` cheklovi cheksiz siklning oldini oladi.

So'rov q: *"Sharh: 'Xizmat juda tez va qulay bo'ldi.' Hissiyotini ayt."*

Mavjud vositalar: sentiment_classify, retrieve_docs, summarize.

t	Tanlangan harakat a_t	Kuzatuv o_t
1	sentiment_classify(text=review)	{label: ijobiy, score: 0.91}
2	is_final	Yakuniy javob qaytariladi

Izoh

Bu misolda agent uchta vosita ichidan faqat sentiment_classify ni tanladi; retrieve_docs va summarize kerak bo'lmadi.

$h_1 = [(a_1, o_1)]$ tarixga qo'shilgach, agent natijani **yetarli** deb topadi va $\hat{y} = \text{"Sharh ijobiy"}$ javobini qaytaradi (score = 0.91 bu yerda namunaviy ishonch bahosi).

Xulosa: ba'zan bitta harakat kifoya qiladi; murakkab vazifalarda esa ReAct sikli bir necha qadam davom etadi.

Kod $\leftrightarrow$ formula: ReAct sikli

```
def react(q, tools, llm, max_steps=5):
    h = [] #  $h_0 = []$ 
    for t in range(1, max_steps + 1):
        a = llm.decide(q, h) #  $a_t = LLM(q, h_{t-1})$ 
        if a.is_final:
            return a.answer #  $y_{hat}$ 
        tool = tools[a.name]
        o = tool(**a.args) #  $o_t = tool(a_t)$ 
        h.append((a, o)) #  $h_t = h_{t-1} \parallel [(a_t, o_t)]$ 
    return llm.finalize(q, h) # max_steps: xavfsiz to'xtash
```

Kod $\rightarrow$ formula

Kod	Ma'nosi
$h = []$	$h_0 = []$
<code>llm.decide</code>	$a_t = LLM(q, h_{t-1})$
<code>a.is_final</code>	$\hat{y}$ qaytadi
<code>tool(...)</code>	$o_t = tool(a_t)$
<code>h.append</code>	$h_t = h_{t-1} \parallel [(a_t, o_t)]$

Izoh

Bu minimal pedagogik kod. Real agentda tool nomi, argument sxemasi, xatoliklar va ruxsatlar ham tekshiriladi.

Keyingi bo'limlarda shu sikl tool/API va LangChain orqali tartiblanadi.

ReAct tuzog'i: cheksiz sikl

Muammo: sikl to'xtamasligi mumkin

Agent `is_final` holatiga kelmasa yoki bir xil harakatni qayta-qayta tanlasa, ReAct sikli davom etaveradi. Natijada **vaqt**, **token** va **API xarajati** ortadi.



Yechim: nazorat qo'yish

- 1 **max_steps**: qadamlar sonini cheklash.
- 2 **Budjet**: vaqt, token yoki API chaqiruv limitini belgilash.
- 3 **Takrorlanishni aniqlash**: bir xil tool va argumentlar qaytalansa, siklni to'xtatish yoki strategiyani almashtirish.

Agent erkin ishlaydi: u doim **to'xtash sharti**, **xavfsizlik limiti** va **loglash** bilan boshqariladi.

Prezentatsiya rejasi

- 1 Kirish: LLM dan agentga
- 2 Agent va ReAct sikli
- 3 Vositalar va funksiya chaqirish**
- 4 NLP modelini API qilish
- 5 Xavfsizlik va LangChain
- 6 Xulosa va keyingi ma'ruzaga ko'prik

Vosita (tool) nima va model qanday tanlaydi?

Tool — agent chaqira oladigan funksiya yoki xizmat. U odatda uch qism bilan beriladi:

`tool = (name, description, schema).`

Qism	Vazifasi
name	tool nomi, masalan summarize;
description	tool nima qilishini tabiiy tilda tushuntiradi;
schema	qanday argumentlar kerakligini belgilaydi, masalan text: string.

Intuitsiya: model foydalanuvchi so'rovini tool tavsiflari bilan moslashtirib, eng mos toolni tanlaydi.

Tanlash mexanizmi

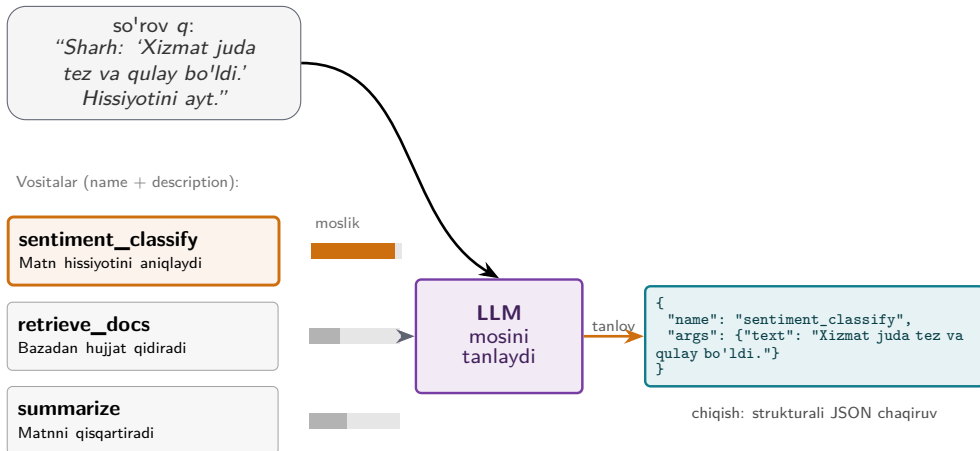
Tool tavsiflari modelga taqdim etiladi. Model javob sifatida odatda **strukturali chaqiruv** yaratadi: tool nomi va argumentlar.

Muhim aniqlik

LLM toolni **tanlaydi**; toolni esa agent dasturi yoki framework **bajaradi**. Natija keyingi qadam uchun kuzatuv sifatida qaytadi.

Tool tanlash jarayoni

$$\text{tool}_i = (\text{name}_i, \text{description}_i, \text{schema}_i), \quad a_t = (\text{name}^*, \text{args}^*) = \text{LLM}(q, \{\text{tool}_i\}_{i=1}^m)$$



Intuitsiya: model so'rovni tool tavsiflari bilan solishtirib, eng mos vositani tanlaydi. **Muhim:** LLM JSON chaqiruvni yaratadi, toolni esa agent dasturi bajaradi.

JSON schema: nega strukturali chiqish kerak?

JSON (*JavaScript Object Notation*) — kalit-qiyamat juftliklaridan iborat matnli ma'lumot formati.
Schema — JSONda qaysi maydonlar bo'lishi, ularning turi va majburiyligini belgilovchi qoida.

Erkin matn: ishonchsiz

“Menimcha sentiment_classify ni ishlatsa bo'lar...”

Bunday javobni dastur avtomatik ajratishi mumkin, lekin **barqaror emas**: nom, argument va format har safar turlicha chiqishi mumkin.

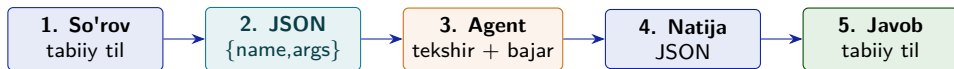
Strukturali JSON: ishonchliroq

```
{  
  "name": "sentiment_classify",  
  "args": {"text": "Xizmat yaxshi"}  
}
```

Kod name va args maydonlarini aniq o'qiydi, schema bo'yicha tekshiradi va mos toolni chaqiradi.

Strukturali chiqish agentni ishonchli avtomatlashtirishga yordam beradi: LLM **tool chaqiruvini JSON sifatida yaratadi**, agent dasturi esa uni **tekshiradi** va keyin vositani bajaradi.

Funksiya chaqirish (function-calling) oqimi



- **Tabiiy til so'rovi:** foydalanuvchi vazifani oddiy tilda beradi.
- **JSON chaqiruv:** LLM qaysi tool va qanday argument kerakligini strukturali ko'rinishda chiqaradi.
- **Tekshirish va bajarish:** agent dasturi JSONni schema bo'yicha tekshiradi, toolni chaqiradi va natijani oladi.
- **Yakuniy javob:** LLM tool natijasini foydalanuvchiga tushunarli tabiiy tilda izohlaydi.

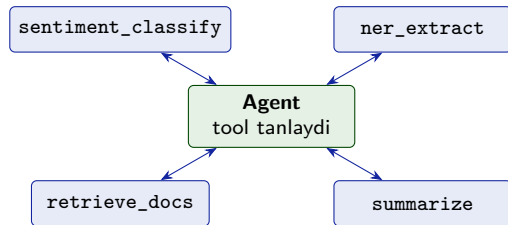
Asosiy farq: LLM tool chaqiruvini **taklif qiladi**; bajarishni agent/framework amalga oshiradi.

NLP modellari — agent vositalari sifatida

Oldingi ma'ruzalardagi NLP komponentlarini agent **vosita (tool)** sifatida chaqirishi mumkin:

- `sentiment_classify` — hissiyot tahlili (L1/L2/L13).
- `ner_extract` — nomli birliklarni ajratish (NER, L10).
- `retrieve_docs` — hujjat qidirish, RAG retriever qismi (L14).
- `summarize` — matnni qisqartirish / umumlashtirish (L12).

Har bir tool bitta aniq vazifani bajaradi; agent esa vazifaga qarab keraklisini tanlaydi.



Asosiy g'oya

Agent hamma ishni bitta model bilan qilmaydi: u kichik NLP xizmatlarini **tool** sifatida chaqirib, ularning natijasini birlashtiradi.

Endi shu toollar qanday qilib **API** orqali chaqirilishini ko'ramiz.

Mashq: qaysi vosita (tool) chaqiriladi?

Savol

So'rov: *"Ushbu uzun maqolani qisqacha aytib ber."*

Mavjud vositalar: `sentiment_classify`, `retrieve_docs`, `summarize`.

Agent qaysi **tool chaqiruvini** yaratishi kerak?

Mashq: qaysi vosita (tool) chaqiriladi?

Savol

So'rov: *"Ushbu uzun maqolani qisqacha aytib ber."*

Mavjud vositalar: `sentiment_classify`, `retrieve_docs`, `summarize`.

Agent qaysi **tool chaqiruvini** yaratishi kerak?

Javob

Tanlanadigan vosita: `summarize`

```
{  
  "name": "summarize",  
  "args": {"text": "maqola matni"}  
}
```

Nega? So'rovdagi *"qisqacha aytib ber"* iborasi `summarize` tavsifiga mos keladi. Hujjat allaqachon berilgan deb olsak, `retrieve_docs` kerak emas; hissiyot so'ralmagani uchun `sentiment_classify` ham tanlanmaydi.

Prezentatsiya rejasi

- 1 Kirish: LLM dan agentga
- 2 Agent va ReAct sikli
- 3 Vositalar va funksiya chaqirish
- 4 NLP modelini API qilish**
- 5 Xavfsizlik va LangChain
- 6 Xulosa va keyingi ma'ruzaga ko'prik

API nima: so'rov va javob protokoli

API (*Application Programming Interface*) — ikki dastur o'zaro qanday gaplashishini belgilovchi shart-noma/protokol. Bu darsda asosan **HTTP endpoint** orqali chaqiriladigan xizmatni nazarda tutamiz: /predict kabi manzilga **request** yuboriladi va **response** olinadi.



Intuitsiya

API — “qanday so'rash va qanday javob olish” qoidasi. Dastur modelning ichki ishlashini bil-maydi; u faqat kelishilgan manzil, format va may-donlar orqali murojaat qiladi.

Modelni API orqali xizmatga chiqarish

NLP model HTTP endpoint ortida ishlaydi. Agent, veb-ilova yoki mobil ilova unga JSON yuborib, JSON natija oladi.

JSON so'rov va javob: POST /predict misoli

So'rov (request):

```
POST /predict
Content-Type: application/json

{
  "text": "mahsulot zo'r"
}
```

Javob (response):

```
200 OK
Content-Type: application/json

{
  "label": "ijobiy",
  "confidence": 0.91
}
```

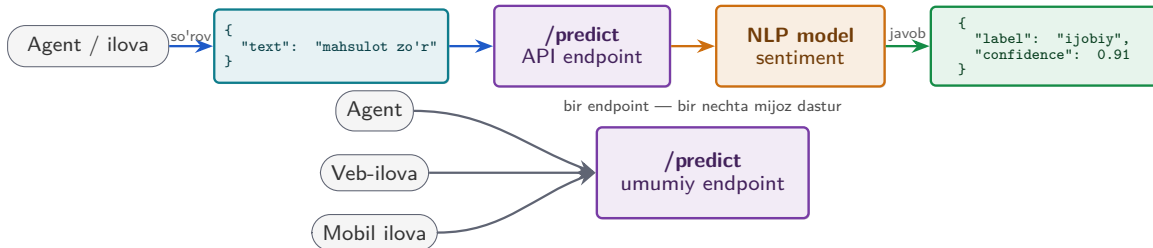
Shartnoma nimani belgilaydi?

POST — ma'lumot yuborib natija so'rash usuli. Bu endpoint text maydonini qabul qiladi; javobda label va modelning namunaviy confidence bahosi qaytadi. Agent yoki ilova shu kelishilgan formatga tayanib modelni chaqiradi.

Muhim: API protokoli barqaror bo'lishi kerak — endpoint, metod, kirish maydonlari va javob maydonlari kutilmaganda o'zgarmasligi lozim.

NLP modelini API orqali xizmatga chiqarish

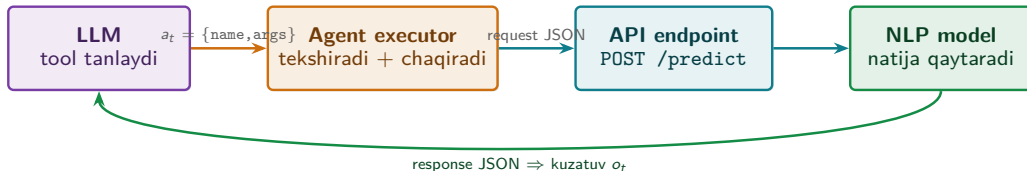
```
POST /predict: request = {text} → response = {label, confidence}
```



Intuitsiya

Model serverda joylashtiriladi; tashqi dasturlar esa modelning ichki kodini bilmasdan, barqaror JSON shartnoma orqali `/predict` endpointini chaqiradi.

Agent + API: tool ko'pincha tashqi xizmatga ulanadi



Asosiy g'oya

Agent uchun tool ko'pincha **API wrapper** bo'ladi: LLM **qaysi tool va qanday argument** kerakligini tanlaydi; agent executor JSONni tekshiradi, HTTP so'rov yuboradi va qaytgan JSONni **kuzatuv** (o_t) sifatida tarixga qo'shadi.

ReAct sikli shu kuzatuv asosida davom etadi: natija yetarli bo'lsa javob qaytariladi, aks holda keyingi tool tanlanadi.

Tashqi APIlar: agentning vositalari

- **Ob-havo API** — joriy ob-havo holatini qaytaradi.
- **Hujjat qidirish** — korxona bazasidan mos hujjat topadi.
- **Sentiment API** — matn hissiyotini baholaydi.
- **Summarization** — uzun matnni qisqartiradi.

Har bir tool aniq tavsif, kirish argumentlari va qaytadigan natija formatiga ega bo'lishi kerak.

Agent qanday foydalanadi?

LLM foydalanuvchi so'rovini tool descriptionlari bilan solishtiradi va kerakli **tool nomi** + **argumentlarni** tanlaydi. Toolni esa agent dasturi chaqiradi.

Muhim aniqlik

Tashqi APIlar agentga joriy, xususiy yoki amaliy ma'lumotdan foydalanish imkonini beradi. Lekin ular doim **ruxsat**, **limit** va **xatoliklarni tekshirish** bilan boshqarilishi kerak.

Prezentatsiya rejasi

- 1 Kirish: LLM dan agentga
- 2 Agent va ReAct sikli
- 3 Vositalar va funksiya chaqirish
- 4 NLP modelini API qilish
- 5 Xavfsizlik va LangChain**
- 6 Xulosa va keyingi ma'ruzaga ko'prik

Agent xavfsizligi: nazoratsiz tool chaqiruvi xavfli

Muammo

Agent har qanday toolni tekshiruvsiz yoki cheksiz chaqira olsa, quyidagi xavflar paydo bo'ladi:

- noto'g'ri amal bajarishi;
- ortiqcha token/API xarajati;
- zararli yoki noto'g'ri kiritmani toolga uzatishi.

Yechim: guardrails

Agentga xavfsizlik chegaralari qo'yiladi:

- 1 **Ruxsat:** qaysi toolni chaqira oladi?
- 2 **Limit:** necha marta va qancha xarajat bilan?
- 3 **Tekshiruv:** argumentlar schema va siyosatga mosmi?

Agent kuchli, chunki u tashqi vositalarni chaqirib **harakat qiladi**. Shuning uchun tool chaqiruvi doim **allowlist**, **schema validation**, **rate/cost limit** va zarur holatda **human approval** bilan boshqariladi.

Guardrails: agent uchun xavfsizlik chegaralari

Guardrails — agentning tool chaqiruvi, argumentlari va natijalarini xavfsiz boshqaruvchi qoidalar va tekshiruvlar to'plami: nimaga ruxsat bor, nimaga yo'q.

Ruxsat va limit

- **Allowlist:** faqat tasdiqlangan tool chaqiriladi.
- **max_steps:** qadamlar soni cheklanadi.
- **Cost/rate limit:** token, vaqt yoki API chaqiruvlar soni nazorat qilinadi.

Tekshiruv va kuzatuv

- **Kiritmani tekshirish:** argumentlar schema va siyosatga mosmi?
- **Logging:** harakat, kuzatuv va xatolar qayd etiladi.
- **Human approval:** xavfli amal uchun inson tasdig'i so'raladi.

Guardrails agentni butunlay xatosiz qilmaydi, lekin noto'g'ri tool chaqiruvi, ortiqcha xarajat va zararli kiritma xavfini kamaytiradi.

Qo'lda yozilgan sikldan frameworkga

Qo'lda yozilgan ReAct sikli

Biz `react(...)` funksiyasida asosiy mantiqni ko'rdik:

- LLM qaror chiqaradi;
- tool chaqiriladi;
- natija tarixga qo'shiladi;
- `is_final` yoki `max_steps` bilan to'xtaydi.

Framework nima beradi?

LangChain kabi frameworklar shu siklni amaliy kodda tashkil qilishni yengillashtiradi:

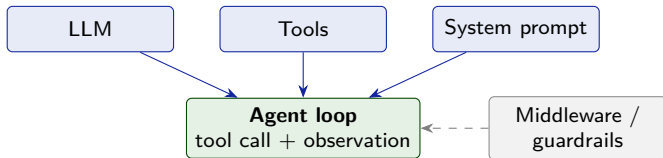
- toollarni ro'yxatdan o'tkazish;
- strukturali tool chaqiruvni ajratish;
- sikl va tarixni boshqarish;
- xatolar va loglarni yuritish.

LangChain ReAct g'oyasini almashtirmaydi; u shu g'oyani **tool**, **LLM**, **tarix** va **boshqaruv sikli** bilan amaliy tizimga yig'ishga yordam beradi. Xavfsizlik chegaralari esa baribir alohida sozlanadi.

Tartib: avval agent siklining mantiqi tushunildi, endi uni framework yordamida qanday yig'ish ko'riladi.

LangChain komponentlari: agent siklini yig'ish

LangChain agent siklini amaliy kodda yig'ishga yordam beradi: LLM, tool ro'yxati, prompt va boshqaruv sikli bitta agent framework ichida ulanadi.



Siz beradigan qismlar

- **LLM** — javob va tool tanlash.
- **Tools** — chaqiriladigan funksiya/API.
- **System prompt** — rol va qoidalar.

Framework bajaradigan ishlar

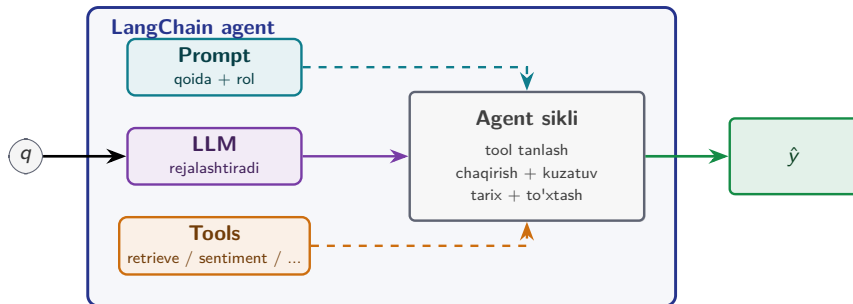
- tool chaqiruvni ajratib bajarish;
- observationni tarixga qo'shish;
- siklni davom yoki tugatish;
- xato, limit va guardrailsni boshqarish.

Eslatma: LangChain ReAct mantiqini almashtirmaydi; uni amaliy agent tizimiga yig'ishni osonlashtiradi.

LangChain agent: ta'rif va intuitiv sxema

LangChain agent — LLM, vositalar (*tools*) va ko'rsatma (*prompt*) asosida ishlaydigan boshqaruv qatlami bo'lib, foydalanuvchi so'rovi uchun **tool chaqiruvi** ↔ **kuzatuv** siklini yuritadi va yakuniy javobni qaytaradi.

$\text{agent} = \text{LangChainAgent}(\text{LLM}, \{\text{Tool}_i\}, \text{prompt}), \quad \hat{y} = \text{agent.invoke}(\{\text{input} : q\})$



Intuitsiya: siz *LLM* +

tools + *prompt* ni berasiz; LangChain esa shu qismlar asosida agent siklini yuritib, yakuniy javobni qaytaradi.

LangChain kod eskizi

```
from langchain.agents import create_agent
# 1) siz berasiz: LLM, tools, system prompt
tools = [retrieve_docs, summarize,
         sentiment_classify]
agent = create_agent(
    model=llm,
    tools=tools,
    system_prompt=(
        "You are an NLP assistant. "
        "Use tools only when needed."
    ),
)
# 2) framework agent siklini yuritadi
result = agent.invoke({
    "messages": [{"role": "user", "content": q}]
})
y_hat = result["messages"][-1].content
```

Kod → ReAct ma'nosi

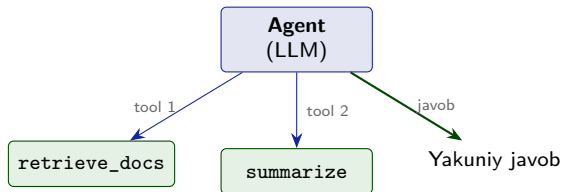
Kod	Ma'nosi
tools	agent chaqira oladigan vositalar ro'yxati
system_prompt	agent roli va qoidalari
create_agent	LLM + tools + promptni agent sikliga ulaydi
invoke	q so'rovni agentga yuboradi va $\hat{y}$ javobni oladi

Eslatma

Bu kod **eskiz**: haqiqiy loyihada model provayderi, API kalitlari, tool schema, xatoliklar va guardrails alohida sozlanadi.

Asosiy g'oya: siz LLM, tools va promptni berasiz; framework tool chaqiruv, kuzatuv va yakuniy javob siklini boshqarishga yordam beradi.

Amaliy loyiha: O'zbek hujjat yordamchisi



Foydalanuvchi so'rasa: agent `retrieve_docs` (bazadan o'zbek hujjatini topadi), keyin kerak bo'lsa `summarize` (qisqartiradi) toollarini chaqiradi; **yakuniy javob** esa tool emas — agentning o'zi qaytaradi. RAG bu yerda **bitta vosita** bo'lib qatnashadi.

Agentdagi tez-tez uchraydigan xatolar

Xato

Noto'g'ri tool: agent vazifaga mos kelmaydigan vositani tanlaydi.

Noto'g'ri JSON: chiqish schema-ga mos kelmaydi.

Cheksiz sikl: `is_final` holati kelmaydi.

Ishonchsiz API: tashqi xizmat xato qaytaradi yoki kechikadi.

Kamaytirish yo'li

Tool descriptionlarini aniq yozish va faqat kerakli tool ro'yxatini berish.

JSON schema validatsiyasi; noto'g'ri formatda qayta so'rash yoki xatoni qaytarish.

`max_steps`, vaqt/token limiti va takror harakatni aniqlash.

Timeout, retry, zaxira xizmat va xatolikni foydalanuvchiga aniq bildirish.

Asosiy g'oya

Agent xatosi odatda modelning o'zidan emas, **tool tanlash**, **format tekshirish**, **sikl nazorati** yoki **API barqarorligi** yetarli boshqarilmagani sababli yuz beradi. Guardrails shu xavflarni kamaytiradi.

Prezentatsiya rejasi

- 1 Kirish: LLM dan agentga
- 2 Agent va ReAct sikli
- 3 Vositalar va funksiya chaqirish
- 4 NLP modelini API qilish
- 5 Xavfsizlik va LangChain
- 6 Xulosa va keyingi ma'ruzaga ko'prik

Sintez: agent — toolsiz LLMdan nimasi bilan farq qiladi?

Jihat	Toolsiz oddiy LLM	Agent
Asosiy vazifa	matn yaratish	rejalashtirish + tool chaqirish
Tashqi manba	bevosita ulanmagan	API, RAG yoki NLP tool orqali ulanadi
Qadamlar	odatda bitta javob	ko'p qadamli sikl: ReAct
Tarix	suhbat konteksti	harakatlar va kuzatuvlar tarixi
Boshqaruv	model javobiga tayanadi	control loop + guardrails bilan boshqariladi

Asosiy g'oya

Agent — LLMga **vositalar**, **qadamlar tarixi** va **boshqaruv sikli** qo'shilgan tizim. U so'rovga qarab kerakli toolni tanlaydi, natijani kuzatadi va keyingi qadamni hal qiladi. RAGning qidiruv qismi (retrieve_docs) agent chaqiradigan vositalardan biri bo'lishi mumkin.

Seminal manba: Yao va boshqalar (2023)

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). **ReAct: Synergizing Reasoning and Acting in Language Models.** *International Conference on Learning Representations (ICLR 2023).*

Nega muhim?

- **Reasoning** (mulohaza yuritish) va **acting** (harakat/tool chaqirish) bitta siklda navbatlashtiriladi.
- Har bir harakatdan keyingi **kuzatuv** modelning keyingi qarorini yangilashga yordam beradi.
- LLM agentlarida **tool ishlatish + kuzatuv + davom ettirish** g'oyasini tushuntirish uchun asosiy konseptual manbalardan biridir.

Muhokama savoli

O'zbek hujjat yordamchisi uchun qaysi guardrails eng zarur: **allowlist**, **max_steps**, **schema validation**, **logging** yoki **human approval**? Nega?

Bugungi maqsadlar bajarildi

- 1 ✓ Agent nima ekani va toolsiz oddiy LLMdan farqi **tushuntirildi**.
- 2 ✓ Agent arxitekturasini **LLM + vositalar (tools) + qadamlar tarixi + boshqaruv sikli** sifatida izohlandi.
- 3 ✓ ReAct sikli a_t , o_t , h_t belgilarida **rasmiy ifodalandi** va qo'lda yozilgan kod bilan bog'landi.
- 4 ✓ Vosita tanlash va funktsiya chaqirish (*function-calling*) JSON orqali **tushuntirildi**.
- 5 ✓ NLP modelini POST /predict endpointi orqali API xizmatga chiqarib, agentga tool sifatida ulash **loyihalashtirildi**.
- 6 ✓ Guardrails va LangChain agent framework roli **ko'rib chiqildi**.

Keyingi ma'ruza (L16): agentdan ishonchli xizmatga

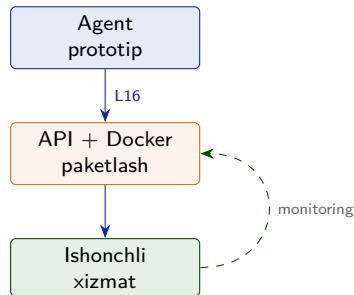
Bugun agentni **prototip darajasida** ko'rdik: ReAct sikli, tool chaqirish, API va guardrails.

L16 da shu tizimni **ishlab chiqarish muhitiga** olib chiqish masalalarini ko'ramiz:

- API sifatida ochish va **Docker** bilan paketlash;
- **monitoring** orqali ishlash holatini kuzatish;
- **CI/CD** orqali xavfsiz yangilash.

L16: MLOps

MLOps — model yoki agentni ishlab chiqarish muhitiga joylashtirish, versiyalash, kuzatish va ishonchli boshqarish amaliyotlari.



L15: agent **qanday ishlaydi**.
L16: agent **qanday ishonchli xizmatga aylantiriladi**.

Maqsad: ishonchlilik, masshtablash va kuzatib borish.

- 1 Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023.
- 2 Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. NeurIPS 2023.
- 3 Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS 2022.
- 4 LangChain rasmiy hujjatlari: *Agents*, *create_agent*, *tools* va *agent loop*.
- 5 OpenAI rasmiy hujjatlari: *Function calling* va *Structured Outputs*.

Eslatma: LangChain va OpenAI API tez yangilanadi; amaliy kod yozishda rasmiy hujjatlarning joriy versiyasini tekshirish kerak.

Ilova S1: ReAct va function-calling farqi

Function-calling — tool chaqiruvni **strukturali formatda** chiqarish mexanizmi. **ReAct** — tool chaqiruvlarni **mulohaza + kuzatuv** bilan bog'laydigan ko'p qadamli agent sikli.

Jihat	Function-calling	ReAct
Asosiy savol	Toolni qanday formatda chaqiramiz?	Toolni qachon va nega chaqiramiz?
Chiqish	JSON: name + args	Harakat a_t , kuzatuv o_t , tarix h_t
Ko'lam	Bitta tool chaqiruvi bosqichi	Ko'p qadamli sikl: fikr $\rightarrow$ harakat $\rightarrow$ kuzatuv
Vazifasi	Kod aniq o'qiydigan strukturali chaqiruv yaratadi	Kuzatuvga qarab keyingi qarorni yangilaydi

Birga ishlashi

Amalda ReAct siklidagi har bir harakat

$$a_t = \{\text{name}, \text{args}\}$$
ko'rinishidagi function-calling JSON bo'lishi mumkin. Demak, ular raqib tushuncha emas: **function-calling format beradi, ReAct esa siklni boshqaradi.**

Ilova S2: tool xatosida qayta urinish (retry)

Tashqi API vaqtinchalik xato qaytarsa (*timeout*, 5xx server xatosi, rate limit), agent darhol taslim bo'lmashligi mumkin. Lekin noto'g'ri argument yoki ruxsat xatosida retry qilish foydasiz.

Xato turi

- **Vaqtinchalik xato:** timeout, 5xx, rate limit.
- **Doimiy xato:** noto'g'ri schema, 400/403, ruxsat yo'lq.
- **Xavfli amal:** qayta urinishdan oldin inson tasdig'i kerak.

Yechimlar

- **Retry:** 2–3 marta qayta urinish.
- **Backoff:** urinishlar orasidagi kutishni oshirish.
- **Fallback:** zaxira tool yoki ehtiyot javob.

Retry faqat **idempotent** yoki xavfsiz tool chaqiruvlarda qo'llanadi. Ya'ni bir xil so'rovni qayta yuborish zararli yon ta'sir keltirmasligi kerak. To'liq monitoring va barqaror deploy mexanizmlari L16 — MLOps mavzusida ko'riladi.